

Document no: P1404r1

Date: 2019-06-16

Authors: Andrzej Krzemiński, Tomasz Kamiński

Reply-to: akrzemi1 (at) gmail (dot) com

Audience: EWG, LEWG

`bad_alloc` is not out-of-memory!

Paper [\[0709R2\]](#) (“Zero-overhead deterministic exceptions: Throwing values”), in its optional part 4.3 (about heap exhaustion), makes an implicit claim that throwing `std::bad_alloc` is synonymous with heap exhaustion and running out of virtual memory; then it builds the optional parts of the proposal based on this claim.

In this paper we show that there are conditions in the program reported through `std::bad_alloc` that do not represent out-of-memory. We also show that there are cases of throwing `std::bad_alloc` that are easily reproducible and testable, and recoverable from.

History

R0 -> R1

Described more cases where `bad_alloc` can be used effectively.

per-allocation limits

Throwing `std::bad_alloc` represents a failure to process a given allocation request for *any* reason. One such reason is that the requested memory size exceeds a per-allocation limit specified in the system for a given program. Consider the following program that tries to allocate a huge chunk of memory:

```
#include <iostream>
#include <vector>
#include <string>

int main()
try {
    std::vector<char> v (1024 * 1024 * 1024); // huge allocation
    std::cout << "OK" << std::endl;
}
catch (std::exception const& e) // bad_alloc handled as any other exception
{
    std::vector<char> s {'E', 'R', 'R', 'O', 'R', ':', ' '}; // reasonable
    allocation
}
```

```
std::cout << std::string(s.begin(), s.end()) << e.what() << std::endl;
}
```

Now, in Linux, we set a limit on a single virtual memory allocation with `ulimit -S -v 204800`, and we run the program. It throws, catches and reports `bad_alloc` elegantly. It even allocates memory while handling the exception. This is all fine because the heap is far from being exhausted. It is only one particular allocation that failed for a particular reason – in our case it was a limit on a single allocation.

This example disproves the claim that heap allocation failure cases are difficult to test.

No room for a new big block

Sometimes, there is a lot of fragmented RAM available, but no single fragment to accomodate a particularly big chunk.

Here is the situation with one of the programs in our company. We need to efficiently store huge data structures in RAM, with lots of pointers, and we have, say, 64 GB of RAM on the machines. In order to address this 64GB we require 32 + 4 bits in a pointer, the remaining 28 bits would be a waste. So, in order to avoid waste, we build the apps in 32-bit mode: this way we are able to address 4GB memory with 32-bit pointers and if we run 16 instances on one box, we are able to address the required 64GB of RAM. Of course, the numbers are oversimplified, but everyone should get the picture. Because we are using 32-bit applications we may get allocation failures not because we run out of virtual memory, but because we run out of addressable space. And this is reported cleanly by throwing `std::bad_alloc` on Linux systems: this is independent of any memory overcommitment strategy.

It happens that while processing some customer requests huge amount of data is generated (which we cannot easily predict ahead of time from just reading the request), and a huge allocation is performed. There is still lots of heap memory available, but because memory is fragmented there is no single chunk to be found capable of storing the required amount of data. This is neatly reported by throwing `std::bad_alloc`. The stack is unwound and the customer request is reported as failed, but the program still runs and is still able to allocate heap memory (even during the stack unwinding).

Custom allocators

Custom allocators report failure to provide requested memory through `std::bad_alloc`. A custom allocator can fail for other reasons than heap exhaustion. A custom allocator can preallocate a chunk of heap memory and distribute it to its callers. When the preallocated memory is run out of, the allocator signals failure – with `std::bad_alloc` – but it is a *local* failure: heap still contains plenty of free memory. Also, some implementations of `Allocator` concept – like “stack allocator” – need not use heap at all; yet they still need to report failure to allocate, and `std::bad_alloc` may be the only choice.

Thus, the argument in [\[0709R2\]](#) that “trying to report and handle heap exhaustion (OOM) by throwing a heap-allocated exception is inherently problematic” does not apply: we are not necessarily reporting any heap exhaustion with `std::bad_alloc`; heap can still be used fine.

Custom allocators can also be used for testing: they can be configured to throw `std::bad_alloc` in predictable places. This makes unit-testing of allocation failure handling in components easy.

Not every program needs to be portable to any platform

Some platforms may unconditionally perform memory overcommitment and on those platforms it is not possible for an implementation to guarantee C++ conformance. However, not every program is written for such platforms. Not every C++ program is written with 100% portability to any platform in mind. Some programs are only meant to be executed on Linux servers. We may then disable memory overcommitment, or use `rlimit` to set a limit on the heap memory allowed for a given program, and programs will report heap allocation failure in a conformant way.

Non-implementability on some platforms is a strong argument. But so is the argument that currently programs on other platforms use the behavior mandated by C++ successfully to implement robust programs: they would be now compromised.

Memory allocation failures vs other resource-related failures

The arguments brought in [\[0709R2\]](#) in favour of not handling memory allocation failures via exceptions — difficult to test, often not tested and therefore handled incorrectly — apply equally well to any other system resource-related failures. Do you use `std::condition_variable` in your codebase? It can throw `errc::resource_unavailable_try_again` from constructor (if some non-memory resource limitation prevents initialization). Is it easy to reproduce this situation? Are you testing this scenario? If not, your code handles it incorrectly. Does this mean a failure to allocate resources necessary to create a `std::condition_variable` should call `std::terminate()` instead of throwing an exception?

What about running out of file descriptors? Is it easy to test? ...

Of course, memory allocation failure is more special in the sense that one usually needs to use the same resource while handling exceptions. But this can be managed safely, as we have shown above. Also, once throwing by value is in place there will be no need to allocate anything while handling exceptions.

Alternatives proposed are unacceptable

The alternative proposed in [\[P0709r2\]](#) that our application should switch to using `try_push_back()` (as proposed in [\[P0132r1\]](#)) or `new (std::nothrow)` instead are not satisfactory: this means we would have to implement manual error handling and propagate it through all the layers in our application. This is practically impossible:

1. We would now have to manually convey the Boolean information about allocation failure through dozens of layers of stack: much like error codes are reported in C, which is – as we know – error prone.

2. We would not be able to report allocation failures from constructors. We allocate memory not only with explicit calls to `push_back()` but also through copying `std::vector`s and `std::string`s. We would have to fall back to two-phase initialization, which is – as we know – error prone.
3. If allocation failure can occur in an operator, it should be replaced with a named function. This is not a frequent case, but this would make the code longer.

Ironically, these three above problems are also used as the motivation for [\[P0709r2\]](#) and exceptions in general, yet what [\[P0709r2\]](#) will force us to do is to replace exception handling with manual error code propagation in our program.

[\[P0709r2\]](#) also suggests that the default `new_handler` should be changed to call `std::terminate()`, and we would have to manually call `std::set_new_handler()` to get a throwing behavior. This is acceptable; but how are goals in [\[P0709r2\]](#) achieved with this? Each `std` function that potentially allocates memory needs to remain potentially throwing, as we never know who and when will call `std::set_new_handler()`.

Separating out-of-memory from other allocation errors

It should be possible to tell apart out-of-memory from other allocation failures reported through `std::bad_alloc`, at least in the default allocator. One way is to provide a program-wide constant which represents the memory allocation threshold. A programmer can set it while building the program, or maybe even later, when the program is run. If memory allocation fails for whatever reason and the memory size requested is smaller than the threshold, it means “we cannot allocate even tiny amount of memory, we will likely not be able to even unwind the stack”, this can be called heap exhaustion; otherwise (if the caller requested for more memory than the threshold), it may mean an unusually big allocation, which cannot be interpreted as heap exhaustion. Under such distinction, it would be acceptable for us to treat out-of-memory as a fatal error.

However, this solution would not work for custom allocators whose “local” failure tells us nothing about the state of the heap.

Also, even in this case memory allocating functions would have to be marked as potentially throwing, and the goal indicated by the optional part of [\[P0709r2\]](#) of “STL functions almost never throw” is still compromised.

Recommendation

[\[P0709r2\]](#) proposes two significant features:

1. being able to throw cheaply (by value),
2. being required to annotate every potentially throwing function with `try`-operator and therewith being able to see in the code any exceptional path.

Our recommendation is to proceed with the first goal and abandon the second. At least drop it from the scope of [\[P0709r2\]](#). Such explicit `try`-operators can be added separately at a later stage. In such alternative future proposal, annotating a potentially throwing function with `try`-operator would be optional, but not putting it on a potentially throwing function could be a compiler warning: compilers do not need the Standard to detect that.

This other proposal could take the path of standardizing a variant of the existing practice: compilers currently offer a no-exceptions mode, where:

1. any `throw` expression is ill-formed,
2. any attempt to start stack unwinding (e.g., from a previously compiled library), calls `std::abort()`.

We could annotate any `std` function that can throw an exception *only* due to an allocation failure from a default allocator and `operator new` with `noexcept(std::no_exceptions_from_new)`, where `std::no_exceptions_from_new` is a `constexpr` constant set to `true` in no-exceptions-from-new mode and to `false` otherwise.

Acknowledgements

Alisdair Meredith, Attila Fehrer and Adam David Alan Martin provided more cases where `bad_alloc` can be used effectively.

References

[\[P0709R2\]](#) – Herb Sutter, “Zero-overhead deterministic exceptions: Throwing values”.

[\[P0132R1\]](#) – Ville Voutilainen, “Non-throwing container operations”.